Longest Increasing Subsequence

Company: Goldman Sachs
Year: Not specified
Difficulty: Medium
Topic: Dynamic Programming, Arrays, Binary Search
Source: LeetCode 300 – Longest Increasing Subsequence

LeetCode 300 – Longest Increasing Subsequence

Problem Description

You are given an integer array nums.

Your task is to find the length of the **longest strictly increasing subsequence**.

A **subsequence** is formed by selecting some elements from the array while maintaining their original order. The selected elements do not need to be next to each other.

A subsequence is **strictly increasing** if every next element is greater than the previous element.

For example:

nums = [10,9,2,5,3,7,101,18]

One longest increasing subsequence is:

[2,3,7,101]

Its length is:

4

Therefore, the answer is 4.

Input Format

- The first line contains a single integer N — the number of elements in the array.
- The second line contains N space-separated integers denoting the array nums.

Output Format

- Print a single integer — the length of the longest strictly increasing subsequence in nums.

Constraints

- 1 ≤ N ≤ 2500
- -10^4 ≤ nums[i] ≤ 10^4

Example 1

Input:

N = 8

nums = [10,9,2,5,3,7,101,18]

Output:

4

Explanation

One longest increasing subsequence is:

[2,3,7,101]

The length is:

4

Example 2

Input:

N = 6

nums = [0,1,0,3,2,3]

Output:

4

Explanation

One possible longest increasing subsequence is:

[0,1,2,3]

Therefore, the answer is:

4

Example 3

Input:

N = 7

nums = [7,7,7,7,7,7,7]

Output:

1

Explanation

The subsequence must be **strictly increasing**.

Since all elements are equal, we cannot select more than one element.

Therefore, the answer is:

1

Approach to Solve This Problem

A straightforward and interview-friendly approach is **Dynamic Programming**.

The main idea is to calculate the longest increasing subsequence **ending at every index**.

For each element nums[i], we check all previous elements nums[j].

If:

nums[j] < nums[i]

then nums[i] can be added after the increasing subsequence ending at j.

So we can extend that subsequence.

DP Definition

Define:

dp[i]

as:

The length of the longest strictly increasing subsequence that ends at index i.

Initially, every element can form a subsequence of length 1 by itself.

Therefore:

dp[i] = 1

for every index.

DP Transition

For every i, check all previous indices j where:

0 <= j < i

If:

nums[j] < nums[i]

then nums[i] can be added after the subsequence ending at j.

Therefore:

dp[i] = max(dp[i], dp[j] + 1)

At the end, the answer is the maximum value in dp.

Example Dry Run

Consider:

nums = [10,9,2,5,3,7]

Initially:

dp = [1,1,1,1,1,1]

For 5:

2 < 5

so:

dp[3] = dp[2] + 1

= 2

For 3:

2 < 3

so:

dp[4] = 2

For 7:

2 < 7

5 < 7

3 < 7

The best subsequence before 7 has length 2, so:

dp[5] = 3

The resulting DP array is:

[1,1,1,2,2,3]

Therefore:

answer = 3

One corresponding subsequence is:

[2,5,7]

Java Solution

```
import java.util.*;

public class Main {

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[] nums = new int[n];

    for (int i = 0; i < n; i++) {
      nums[i] = sc.nextInt();
    }

    int[] dp = new int[n];

    // Every element itself is an increasing subsequence of length 1
    Arrays.fill(dp, 1);

    int maxLength = 1;
```

```
        for (int i = 1; i < n; i++) {

            for (int j = 0; j < i; j++) {

                if (nums[j] < nums[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }

            maxLength = Math.max(maxLength, dp[i]);
        }

        System.out.println(maxLength);
    }
}
```

Solution:

1. **Read the input:** main() reads N, then reads N integers into nums, matching the Input Format.
2. **Create the DP array:** Create an array dp of size N, where dp[i] represents the length of the longest strictly increasing subsequence ending at index i.
3. **Initialize the DP array:** Set every dp[i] to 1 because every individual element can form an increasing subsequence of length 1.
4. **Compare with previous elements:** For every index i, check all previous indices j from 0 to i - 1.

5. **Extend the subsequence:** If nums[j] < nums[i], then nums[i] can be added to the increasing subsequence ending at j. Update dp[i] using dp[i] = Math.max(dp[i], dp[j] + 1).
6. **Track the maximum:** After calculating dp[i], update maxLength with the largest value found so far. This represents the length of the longest increasing subsequence in the entire array.
7. **Print the result:** After processing all elements, print maxLength, matching the Output Format.

Complexity Analysis

- **Time Complexity:** $O(N^2)$ — for every element, we check all previous elements.
- **Space Complexity:** O(N) — the dp array stores the LIS length for every index.